

# Builder Journal

## Productivity & Task Execution Agent

### What I built

I extended my Week 2 RAG application into an agentic system — something that reasons about a request rather than just retrieving and generating text. The agent takes a natural-language message, decides whether a tool is needed, picks the correct one from a registry of 12+ actions, validates the arguments against a schema, executes it against a real MySQL database, and logs every step of that decision process. Sensitive actions — completing or deleting a task — pause and wait for explicit human approval before touching the database.

On top of the core agent loop, I built task management (single or bulk creation from unstructured notes), daily/weekly planning, priority analysis, productivity reporting, a notes system, and LLM-drafted follow-up emails. Everything is browsable through a Flask + Bootstrap frontend: a filterable task board, a plan history, an approvals queue, and a full execution log viewer. I also wrote an automated evaluation suite to check tool selection, approval-gating, and bad-input handling without relying on manual clicking through the UI.

### Most difficult technical problem

The hardest part wasn't any single tool — it was getting the intent analysis step to *reliably* pick the right tool out of 12+ options, and to do it with well-formed arguments every time. Several tools are easy to confuse from natural language alone: "mark the report task done" and "delete the report task" differ by one verb, and `list_tasks` filtered by status can look, from a user's phrasing, a lot like `analyze_priorities`. On top of that, `complete_task` and `delete_task` accept either a `task_id` or a fuzzy `task_title`, which opens the door to ambiguous matches when multiple tasks share similar names.

### How I solved it

I tightened the tool selection prompt in `intent_analysis.py` with explicit per-tool descriptions and a few contrastive examples ("complete" vs "delete" vs "list"), rather than relying on the tool names alone to carry meaning. I leaned on `validator.py` as a hard backstop — even if the LLM picks the right tool, a malformed or missing field gets caught before execution instead of failing deep inside a tool function. For the fuzzy-title problem, I added an explicit ambiguous-match path: if more than one task matches a given title closely enough, the tool returns the candidates instead of guessing, and the controller asks the user to disambiguate rather than approving a possibly-wrong deletion.

### Tool-calling errors observed

- The agent occasionally selected `list_tasks` when the actual intent was closer to `analyze_priorities` — both surface "what's important right now," and the phrasing overlap tripped the intent classifier.
- Malformed `due_date` values showed up when task details were extracted from free text via `create_tasks_from_text` — relative dates like "by Friday" didn't always resolve cleanly to `YYYY-MM-DD`.
- A couple of early runs attempted `complete_task` with a `task_title` that didn't match anything (typo or paraphrase), which the validator and the `TASK_NOT_FOUND` error path caught before anything reached the database.

## Agent behavior that surprised me

The agent was more proactive than I expected with bulk extraction — given a block of messy meeting notes, it would infer several distinct tasks I hadn't explicitly asked it to create, without being told "extract tasks from this." That's useful, but it also means the extraction step needs its own guardrails so it doesn't overreach on text that wasn't meant to produce tasks at all (e.g. a note that's just a reminder to self, not a to-do list).

## What failed during testing

The automated test suite caught a few real gaps: unrelated, conversational queries occasionally still triggered a tool call instead of a plain response, which the evaluation criteria specifically flag as a failure mode. There was also a case where an ambiguous title match should have paused for clarification but instead proceeded against the first candidate — a reminder that "approval required" and "disambiguation required" are two different guardrails and both need to be enforced, not just the one I built first.

## What I would redesign

I'd move approval-gating logic out of individual tool functions (`complete_task`, `delete_task`) and into the tool registry itself, so "this tool requires approval" is a declared property checked in one place rather than duplicated per tool — safer as the tool count grows. I'd also add a dedicated disambiguation step as a first-class part of the pipeline (not a special case inside two tools), since any tool that accepts a fuzzy identifier has the same problem. Finally, I'd add retry/backoff around the OpenRouter calls — a transient LLM failure currently surfaces as a hard error rather than a quiet retry.

## What I learned about agent reliability

Reliability here isn't really about the LLM being "smart enough" — it's about how much the surrounding system refuses to trust the LLM's output blindly. Every place I added a hard check (schema validation, approval gating, ambiguous-match detection) turned an unpredictable LLM decision into a predictable, testable one. The execution log turned out to matter more than I

expected too — most of the bugs I found, I found by reading the trace after the fact, not by watching the chat in real time.

## Goals for Week 4

- Add authentication so the system isn't single-user/localStorage-scoped.
- Move approval-gating to the tool registry level, as above.
- Expand the automated evaluation suite to cover the ambiguous-match and unrelated-query failure cases explicitly.
- Add a calendar-integration tool as a first real external-service tool, to test the registry pattern beyond internal MySQL-backed tools.
- Deploy beyond localhost.